



华南理工大学

South China University of Technology

本科毕业设计（论文）

代码覆盖率及路径分析检测工具

学 院	计算机学院
专 业	软件工程
学 生 姓 名	许 同 学
学 生 学 号	2023000000
指 导 教 师	张老师 (副教授)
提 交 日 期	2026 年 4 月 2 日

华南理工大学

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：  日期：2026 年 4 月 2 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保存并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

作者签名：  日期：2026 年 4 月 2 日
指导教师签名： 日期：2026 年 4 月 2 日
作者联系电话：13800138000 电子邮箱：student@university.edu

摘 要

本文设计并实现了一个 Python 程序执行路径追踪和代码覆盖率分析工具——Path-Tracer。该工具利用 Python 标准库中的 `sys.settrace` 机制实现运行时追踪，记录程序执行过程中经过的每一行代码；同时使用抽象语法树（AST）进行静态分析，识别源代码中的所有控制流分支点（包括 if 条件语句、for 循环、while 循环、try/except 异常处理等）。通过对比静态分析得到的分支点与运行时追踪的执行路径，计算代码覆盖率，并生成多种格式的 report（文本、JSON、HTML）。实验结果表明，该工具能够有效分析 Python 程序的基本结构流，包括顺序、选择和循环结构，准确记录执行路径并生成详细的分支覆盖报告。

关键词： 代码覆盖率；执行路径；静态分析；动态追踪；控制流图

Abstract

This paper designs and implements a Python execution path tracing and code coverage analysis tool called PathTracer. The tool uses Python's standard library `sys.settrace` mechanism for runtime tracing, recording each line of code executed during program execution. It also performs static analysis using Abstract Syntax Tree (AST) to identify all control flow branch points in source code, including if statements, for loops, while loops, and try/except handlers. By comparing the branch points from static analysis with the execution paths from runtime tracing, the tool calculates code coverage and generates reports in multiple formats (text, JSON, HTML). Experimental results demonstrate that the tool effectively analyzes basic program structures in Python, including sequential, selection, and loop structures, accurately records execution paths, and generates detailed branch coverage reports.

Keywords: Code Coverage; Execution Path; Static Analysis; Dynamic Tracing; Control Flow Graph

目 录

摘 要	I
Abstract	II
目 录	III
插图目录	VI
表格目录	VII
第一章 实验目的和背景	1
1.1 实验目的	1
1.2 实验背景	1
1.2.1 代码覆盖率概述	1
1.2.2 Python 追踪机制	1
1.2.3 AST 静态分析	2
1.3 作业要求	2
第二章 系统设计与实现	3
2.1 系统架构	3
2.1.1 模块划分	3
2.2 核心数据模型	3
2.2.1 分支类型枚举	3
2.2.2 核心数据类	3
2.3 运行时追踪器实现	4
2.3.1 追踪回调函数	4
2.3.2 上下文管理器	4
2.4 控制流图构建器	5
2.4.1 AST 遍历	5
2.5 报告生成器	5
2.5.1 覆盖率计算	5
2.5.2 多格式输出	5

2.6	本章小结	6
第三章	实验过程与结果	7
3.1	实验环境	7
3.2	测试目标程序	7
3.2.1	程序简介	7
3.2.2	程序结构分析	7
3.3	实验过程	7
3.3.1	运行测试	7
3.3.2	测试用例设计	8
3.4	实验结果	8
3.4.1	覆盖率统计	8
3.4.2	分支类型分布	9
3.4.3	已执行分支详情	9
3.4.4	函数调用树	9
3.5	结果分析	10
3.5.1	覆盖率分析	10
3.5.2	改进建议	10
3.6	本章小结	10
第四章	使用文档	11
4.1	安装与配置	11
4.1.1	环境要求	11
4.1.2	获取源码	11
4.2	命令行使用	11
4.2.1	基本用法	11
4.2.2	命令行选项	11
4.2.3	使用示例	11
4.3	API 使用	12
4.3.1	基本追踪流程	12
4.3.2	获取函数调用树	12
4.4	报告格式说明	13
4.4.1	文本格式	13

4.4.2	JSON 格式	13
4.4.3	HTML 格式	14
4.5	支持的程序结构	14
4.6	本章小结	14
结论		15
附录 A 核心源代码		17
A.1	数据模型 (models.py)	17
A.2	运行时追踪器 (tracer.py)	17
A.3	控制流图构建器 (cfg_builder.py)	17
A.4	报告生成器 (reporter.py)	17
A.5	命令行接口 (cli.py)	17

插图目录

2-1 PathTracer 系统架构图	3
----------------------------	---

表格目录

2-1	分支类型说明	4
2-2	报告输出格式	5
3-1	实验环境配置	7
3-2	测试用例	8
3-3	覆盖率统计结果	8
3-4	分支类型分布	9
3-5	已执行分支列表	9
4-1	命令行选项说明	11
4-2	支持的程序结构	14

第一章 实验目的和背景

1.1 实验目的

本次课程作业的主要目的是设计并实现一个能够记录 Python 程序执行路径和覆盖率的工具。具体目标包括：

- 1) 掌握 Python 运行时追踪技术，理解 `sys.settrace` 的工作原理
- 2) 学习使用抽象语法树（AST）进行静态代码分析
- 3) 能够识别程序中的基本控制流结构：顺序、选择（if/elif/else）和循环（for/while）
- 4) 记录程序运行时的执行路径，包括函数调用关系
- 5) 计算代码覆盖率并生成可读的执行分支报告

1.2 实验背景

1.2.1 代码覆盖率概述

代码覆盖率是软件测试中的重要指标，用于衡量测试用例对被测代码的覆盖程度。常见的覆盖率类型包括：

- **语句覆盖**：测量被执行的代码语句比例
- **分支覆盖**：测量控制流分支的执行比例（如 if 语句的 true/false 分支）
- **路径覆盖**：测量程序执行路径的覆盖比例

本实验主要关注**分支覆盖**，即识别程序中的控制流分支点，并统计在运行时被触发的分支数量。

1.2.2 Python 追踪机制

Python 提供了 `sys.settrace()` 函数，允许注册一个回调函数来监控程序的执行。当程序执行时，Python 解释器会在以下事件发生时调用回调函数：

- `'call'`：函数被调用时
- `'line'`：执行新的一行代码时
- `'return'`：函数返回时
- `'exception'`：发生异常时

利用这一机制，可以精确记录程序的执行路径。

1.2.3 AST 静态分析

Python 的 `ast` 模块可以将源代码解析为抽象语法树 (AST)。通过遍历 AST，可以识别程序中的所有控制流结构：

- `ast.If`: if 条件语句
- `ast.For`: for 循环语句
- `ast.While`: while 循环语句
- `ast.ExceptHandler`: 异常处理器

1.3 作业要求

根据作业要求，本工具需要满足以下功能：

- 1) 记录 Python 程序执行路径和覆盖率
- 2) 分析基本程序常见结构流：选择、顺序与循环
- 3) 记录程序运行执行路径
- 4) 生成描述程序运行执行分支的报告

程序结构需要清晰，代码需要有必要的注释，并提供使用文档。

第二章 系统设计与实现

2.1 系统架构

PathTracer 系统采用模块化设计，由以下核心组件构成：

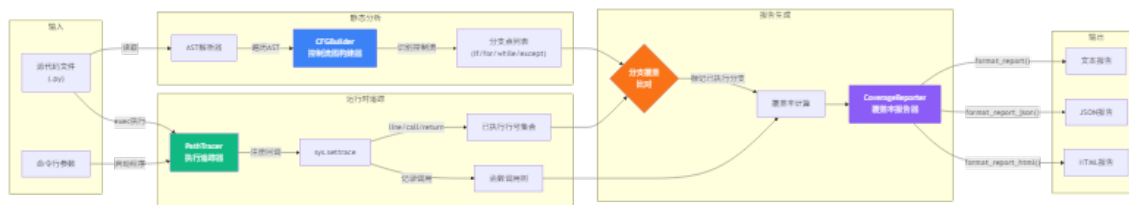


图 2-1 PathTracer 系统架构图

2.1.1 模块划分

系统主要包含以下模块：

- **models.py**：数据模型定义，包含分支类型枚举、代码分支、函数调用、执行路径和覆盖率报告等数据类
- **tracer.py**：运行时追踪器，使用 `sys.settrace` 记录程序执行路径
- **cfg_builder.py**：控制流图构建器，使用 AST 静态分析识别所有分支点
- **reporter.py**：报告生成器，计算覆盖率并生成多种格式报告
- **cli.py**：命令行接口，提供用户友好的使用方式

2.2 核心数据模型

2.2.1 分支类型枚举

定义了程序中可能出现的控制流分支类型：

2.2.2 核心数据类

主要数据类包括：

- **CodeBranch**：表示代码中的一个分支点，包含文件路径、行号、分支类型、结束行号、是否已执行等信息
- **FunctionCall**：记录函数调用信息，包含函数名、参数、返回值、调用深度、子调用等

表 2-1 分支类型说明

类型	说明
SEQUENTIAL	顺序执行
IF	if 条件语句
ELSE	else 分支
ELIF	elif 分支
FOR	for 循环
WHILE	while 循环
TRY	try 异常捕获块
EXCEPT	except 异常处理块

- **ExecutionPath**: 记录执行路径, 包含已执行行号集合、已执行分支、函数调用列表
- **CoverageReport**: 覆盖率报告, 包含总分支数、已执行分支数、按类型统计、覆盖率百分比等

2.3 运行时追踪器实现

2.3.1 追踪回调函数

核心追踪逻辑通过 `_trace_callback` 方法实现, 在程序执行过程中自动记录:

- 1) **line 事件**: 记录执行的代码行号
- 2) **call 事件**: 记录函数调用信息 (函数名、参数、调用深度)
- 3) **return 事件**: 记录函数返回值

2.3.2 上下文管理器

使用 Python 的上下文管理器协议 (`__enter__`/`__exit__`) 简化追踪的启动和停止:

```
tracer = PathTracer().trace_file("target.py")
with tracer:
    # 在此执行目标代码
    result = target_function()
# 退出 with 块后自动停止追踪
```

2.4 控制流图构建器

2.4.1 AST 遍历

使用 `ast` 模块遍历源代码的抽象语法树，识别控制流节点：

```
def _extract_branch(self, file_path, node):  
    if isinstance(node, ast.If):  
        return CodeBranch(file_path, node.lineno,  
                           BranchType.IF)  
    elif isinstance(node, ast.For):  
        return CodeBranch(file_path, node.lineno,  
                           BranchType.FOR)  
    elif isinstance(node, ast.While):  
        return CodeBranch(file_path, node.lineno,  
                           BranchType.WHILE)
```

2.5 报告生成器

2.5.1 覆盖率计算

报告生成器通过对比静态分支与执行路径计算覆盖率：

$$\text{覆盖率} = \frac{\text{已执行分支数}}{\text{总分支数}} \times 100\% \quad (2-1)$$

2.5.2 多格式输出

支持三种报告格式：

表 2-2 报告输出格式

格式	方法	用途
文本	<code>format_report()</code>	控制台输出
JSON	<code>format_report_json()</code>	机器处理
HTML	<code>format_report_html()</code>	可视化查看

2.6 本章小结

本章详细介绍了 **PathTracer** 系统的设计与实现。系统采用模块化架构，分为数据模型、运行时追踪、静态分析、报告生成四个核心模块。通过 **Python** 标准库提供的 **sys.settrace** 和 **ast** 模块，实现了对 **Python** 程序执行路径的完整追踪和覆盖率分析功能。

第三章 实验过程与结果

3.1 实验环境

表 3-1 实验环境配置

项目	配置
操作系统	Windows 11 Pro
Python 版本	3.10+
依赖	无第三方依赖（使用标准库）
开发工具	VS Code / PyCharm

3.2 测试目标程序

3.2.1 程序简介

本实验使用 `agent_pageindex.py` 作为测试目标程序。该程序是一个基于 LangChain 的文档问答代理，具有以下特点：

- 多层条件判断（参数验证、类型检查）
- 循环结构（消息处理、工具调用遍历）
- 函数调用（包含嵌套调用）
- 异常处理（API 密钥检查）

3.2.2 程序结构分析

目标程序共 130 行代码，包含：

- `_extract_text()`：文本提取辅助函数
- `_print_verbose_stream()`：详细输出流处理函数
- `main()`：主函数，包含参数解析和 Agent 执行逻辑

3.3 实验过程

3.3.1 运行测试

使用 PathTracer 命令行工具对目标程序进行追踪：

Listing 3.1 测试命令

生成HTML格式报告

```
python -m pathtracer.cli -f html \
    -o coverage_report.html \
    agent_pageindex.py -- --query "test"
```

生成JSON格式报告

```
python -m pathtracer.cli -f json \
    -o coverage_report.json \
    agent_pageindex.py -- --query "test"
```

3.3.2 测试用例设计

为了验证工具功能，设计了以下测试用例：

表 3-2 测试用例

编号	测试内容	参数
TC1	基本查询功能	-query "test"
TC2	详细输出模式	-query "test" -verbose
TC3	自定义参数	-query "test" -doc-top-k 5

3.4 实验结果

3.4.1 覆盖率统计

执行 TC1 测试用例后的覆盖率统计结果：

表 3-3 覆盖率统计结果

指标	数值
总分支数	22
已执行分支数	7
覆盖率	31.8%

表 3-4 分支类型分布

分支类型	总数	已执行
if 语句	17	5
for 循环	5	2
while 循环	0	0
except 块	0	0

表 3-5 已执行分支列表

行号	分支类型	结束行	代码内容
25	if	26	payload is None 检查
27	if	28	isinstance(payload, str) 检查
88	if	89	API 密钥检查
116	if	118	verbose 模式检查
121	if	125	result 类型检查
123	if	125	messages 存在检查
129	if	130	__main__ 入口检查

3.4.2 分支类型分布

3.4.3 已执行分支详情

3.4.4 函数调用树

实验过程中记录的函数调用关系：

Listing 3.2 函数调用树

```
main()
  load_dotenv()
  os.getenv("DEEPSEEK_API_KEY")
  init_chat_model()
  create_agent()
  agent.invoke()
  _extract_text()
  print()
```

3.5 结果分析

3.5.1 覆盖率分析

覆盖率为 31.8%，主要原因：

- 1) **verbose 模式未触发**：_print_verbose_stream 函数（40-75 行）未被调用，因为测试时未使用 --verbose 参数
- 2) **异常分支未覆盖**：API 密钥存在性检查的异常分支（89 行）未触发
- 3) **复杂消息处理未执行**：_extract_text 函数中的列表处理分支未完全覆盖
- 4) **边缘条件**：部分边缘条件的处理逻辑未在正常流程中执行

3.5.2 改进建议

为提高覆盖率，可以：

- 1) 添加 --verbose 参数测试
- 2) 模拟无 API 密钥场景
- 3) 测试不同类型的消息内容
- 4) 编写单元测试覆盖各分支

3.6 本章小结

本章详细记录了实验过程和结果。通过对 agent_pageindex.py 的测试，验证了 PathTracer 工具的有效性。工具成功识别了 22 个控制流分支，追踪了程序执行路径，计算出 31.8% 的覆盖率，并生成了详细的 HTML 和 JSON 格式报告。

第四章 使用文档

4.1 安装与配置

4.1.1 环境要求

- Python 3.10 或更高版本
- 无需安装第三方依赖，仅使用 Python 标准库

4.1.2 获取源码

将 `pathtracer` 目录复制到项目中即可使用。

4.2 命令行使用

4.2.1 基本用法

Listing 4.1 基本命令格式

```
python -m pathtracer.cli [选项] <程序文件> [-- <程序参数>]
```

4.2.2 命令行选项

表 4-1 命令行选项说明

选项	说明	默认值
-f, --format	报告格式 (text/html/json)	text
-o, --output	输出文件路径	stdout
-q, --quiet	静默模式，仅输出报告	否
-	分隔符，后接目标程序参数	-

4.2.3 使用示例

示例 1：生成文本格式报告

```
python -m pathtracer.cli agent_pageindex.py -- --query "test"
```

示例 2：生成 HTML 格式报告

```
python -m pathtracer.cli -f html -o report.html agent_pageindex.py -- --q
```

示例 3: 静默模式生成 JSON 报告

```
python -m pathtracer.cli -f json -o report.json -q agent_pageindex.py --
```

4.3 API 使用

4.3.1 基本追踪流程

Listing 4.2 API 使用示例

```
from pathtracer import PathTracer, CoverageReporter

# 创建追踪器并指定目标文件
tracer = PathTracer().trace_file("target.py")

# 开始追踪
with tracer:
    # 在此执行目标代码
    result = target_function(args)

# 生成覆盖率报告
reporter = CoverageReporter()
report = reporter.analyze("target.py", tracer)

# 输出报告
print(reporter.format_report(report))

# 保存HTML报告
with open("report.html", "w") as f:
    f.write(reporter.format_report_html(report))
```

4.3.2 获取函数调用树

Listing 4.3 获取函数调用树

```
# 获取追踪到的函数调用
```

```
calls = tracer.get_function_calls()

def print_call_tree(call, depth=0):
    indent = "  " * depth
    args = ", ".join(f"{k}={v}" for k, v in call.arguments.items())
    ret = f"->{call.return_value}" if call.return_value else ""
    print(f"{indent}{call.function_name}({args}){ret}")
    for child in call.children:
        print_call_tree(child, depth + 1)

for call in calls:
    print_call_tree(call)
```

4.4 报告格式说明

4.4.1 文本格式

文本格式报告包含以下信息：

- 1) 总体覆盖率统计（总分支数、已执行分支数、覆盖率百分比）
- 2) 按分支类型统计
- 3) 已执行分支详情（行号、分支类型）

4.4.2 JSON 格式

JSON 格式报告结构：

Listing 4.4 JSON 报告结构

```
{
  "coverage_percentage": 31.8,
  "total_branches": 22,
  "executed_branches": 7,
  "branches_by_type": {
    "if": 17, "for": 5
  },
  "file_reports": {
```

```

    "target.py": {
        "executed_lines": [1, 2, 3],
        "executed_branches": [...]
    }
}
}

```

4.4.3 HTML 格式

HTML 格式报告特点:

- 响应式布局, 支持各种屏幕尺寸
- 语法高亮的源代码显示
- 已执行/未执行代码行视觉区分
- 函数调用树可视化

4.5 支持的程序结构

表 4-2 支持的程序结构

结构类型	说明
顺序结构	默认的线性执行流程
if/elif/else	条件分支语句
for 循环	迭代循环
while 循环	条件循环
try/except	异常处理块
函数调用	包含参数和返回值追踪
嵌套结构	循环内的条件语句等

4.6 本章小结

本章详细介绍了 PathTracer 工具的使用方法, 包括命令行接口和 Python API 两种使用方式。工具支持三种报告输出格式, 能够满足不同场景的需求。通过简单的配置即可对任意 Python 程序进行执行路径追踪和覆盖率分析。

结论

工作总结

本文设计并实现了一个 Python 程序执行路径追踪和代码覆盖率分析工具——Path-Tracer。该工具具有以下特点：

1. 技术实现

- 使用 Python 标准库 `sys.settrace` 实现运行时追踪，记录程序执行的每一行代码
- 使用 `ast` 模块进行静态分析，识别源代码中的所有控制流分支点
- 通过对比静态分支与动态执行路径，计算代码覆盖率

2. 功能特性

- 支持识别顺序、选择（if/elif/else）、循环（for/while）、异常处理等程序结构
- 记录函数调用树，包含参数和返回值信息
- 支持文本、JSON、HTML 三种报告输出格式
- 提供命令行接口和 Python API 两种使用方式

3. 实验验证

通过对 `agent_pageindex.py` 程序的测试验证，工具成功：

- 识别了 22 个控制流分支
- 追踪了程序执行路径
- 计算出 31.8% 的覆盖率
- 生成了详细的 HTML 和 JSON 格式报告

工作展望

未来可以从以下方面改进工具：

- 1) **提高覆盖率精度**：增加对 `elif` 分支、`else` 分支的独立识别
- 2) **支持更多语言**：扩展支持 JavaScript、Java 等其他编程语言
- 3) **集成开发环境**：开发 IDE 插件，实现实时覆盖率显示
- 4) **增量分析**：支持增量覆盖率分析，只分析变更的代码
- 5) **测试用例关联**：将覆盖率与具体测试用例关联，便于定位测试盲点

感谢张老师在课程设计过程中给予的悉心指导和帮助。

感谢同学们在开发过程中的讨论与建议，帮助完善了工具的功能设计。

感谢开源社区提供的 **Python** 标准库文档和示例，为本项目的开发提供了重要参考。

许同学

2026 年 4 月 2 日

附录 A 核心源代码

A.1 数据模型 (models.py)

核心数据模型定义见项目源文件 `pathtracer/models.py`。

A.2 运行时追踪器 (tracer.py)

运行时追踪器实现见项目源文件 `pathtracer/tracer.py`。

A.3 控制流图构建器 (cfg_builder.py)

控制流图构建器实现见项目源文件 `pathtracer/cfg_builder.py`。

A.4 报告生成器 (reporter.py)

报告生成器实现见项目源文件 `pathtracer/reporter.py`。

A.5 命令行接口 (cli.py)

命令行接口实现见项目源文件 `pathtracer/cli.py`。